Project 2: The Mystery Binary

Himadri Saha

EECE.5821 Computer Architecture

11/11/2026

Part A:

For single-cycle, one clock cycle per instruction is used. This is controlled by the control unit, which takes an instruction and generates the appropriate control signals. For lw x5, 0(x20), the control signal for the register write is active. The two ALU inputs come from register x20 and the immediate respectively. The ALU output is therefore the addition of these two values. The memory is read and the value placed in register x5. For sub x7, x5, x6, both ALU inputs are from the registers and ALU performs subtraction operation with its output stored in register x7. For beq x7, x0, equal, there is no register write. The two ALU inputs are the PC and the immediate. The ALU adds them to get the branch target (0x1c) which is equal to the register value of x7. In sw x6, 4(x20), the ALU adds x20 and the offset to get the address, and memory write is active. No register is written. In jal x1, func, the value of PC+4 is stored in x1, the ALU adds the PC and offset to get the target address (0x30), and the jump is unconditional. In slli x10, x5, 1, the ALU left-shifts x5 by one position, and stores the result in x10. In jalr x0, x1, 0, the target is got by adding x1 and zero (0x24), the value of PC+4 is stored in x0, which discards the value, and the jump is taken. In the second beq x9, x0, equal, the ALU calculates the target address (0x1c). But since x9 is 9, the branch is not taken.

In A2, the ALU does not perform comparison between x7 and x0 in case of a taken branch as it just calculates the branch target through the addition of PC and offset. The Branch unit performs the operation to determine whether the branch is taken through comparison between x7 and x0, and the branch-taken signal determines the PC.

A3 will have a program of 14 instructions (including la as two), therefore, the single-cycle CPI will be 1 and it will take 14 clock cycles to run it. Single-Cycle clock period is 900 ps (sum of the clock periods of all five stages: 200 + 150 + 200 + 250 + 100). Clock period for 5 stages is 270 ps (slowest stage + one pipeline register period: 250 + 20).

Part B:

B1 schedule retires 8 instructions in 12 cycles. The pipeline requires 4 more cycles to fill, since the first instruction takes 5 cycles to go through all five pipeline stages. The program does not have any branches, thus nothing gets flushed. No hazard detection mechanism is considered in this model, therefore no stall cycle is needed as well.

The instruction at IF stage in cycle 4 is slli x9, x8, 2; the instruction at ID stage is sub x8, x7, x5; the instruction at EX stage is add x7, x5, x6; the instruction at MEM stage is addi x6, x0, 3; and the instruction at WB stage is addi x5, x0, 10.

alu_ctrl and alu_op2_ctrl signals are needed in the EX stage, hence they have to go through ID/EX pipeline stage. mem_do_write_ctrl signal is required in MEM stage, hence it has to pass through the EX/MEM pipeline stage. reg_wr_src_ctrl and reg_do_write_ctrl signals are needed

in WB stage, therefore they have to go through MEM/WB pipeline stage. rd should also reach WB stage since the register file should know which register will be written into.

These registers are incorrect for x7, x8, x9, x11, and x12. The correct values are 13, 3, 12, 13, and 26, respectively. The registers x5, x6, and x10 are correct. The value of x8 is 0xFFFFFFF6 since the sub instruction accessed x7 prior to the addition writing to x7. The sub instruction accessed the value of x7 as 0 since the old value was used, and it accessed x5 correctly as 10; thus, x8 = 0 – 10 = -10 = 0xFFFFFFF6.

The program requires 10 nops to work properly. There should be two instructions in-between producer and consumer since producer writes the value into the WB while consumer reads operands from ID stage in the same cycle. With just one instruction in-between, the value has not been written yet. Five dependent holes require two nops each, so using less than that would yield incorrect results. The total number of cycles required for this program is 22: 8 instructions, 10 nops, and 4 fills.

Part C:

The original code b_hazard.s when executed on RV32 5S NO FW returns the right answers from x5 till x12 in 22 cycles, out of which 10 cycles are stall cycles. The Hazard Unit stalls all the dependent instructions by an amount equal to the number of NOPs in b nops.s, and hence both return in 22 cycles.

Each instruction spends a unique number of cycles in ID stage. The first two instructions require 0 cycles of delay and the third one (add x7, x5, x6) 2 cycles of delay. It is delayed due to its dependency on x6 that comes from the second instruction (addi x6). The fourth, fifth, sixth, and eighth instructions spend 2 cycles each in ID due to their dependency on the instruction before them. The seventh one spends 0 cycles since it has no dependencies at all. Thus, 0 + 0 + 2 + 2 + 2 + 0 + 2 + 2 = 10 cycles are required.

During the stall cycle, the hazard unit keeps the PC and the IF/ID registers occupied, thus keeping the current instruction at ID, and no new instruction is fetched by the IF stage. Also, it empties the ID/EX register; this creates a bubble such that nothing is done at EX for the cycle. This bubble has to be created in ID/EX as the current instructions at EX, MEM, and WB have to keep progressing so that the producer completes and writes the result.

The runtime of d_load.s using RV32 5S NO FW is 27 cycles. The instruction set contains 11 instructions, where la is counted as two instructions. Stall cycles include 12 cycles, where each load use costs 2 cycles, while each auipc to addi and addi to lw uses 2 cycles each. The instruction set includes 3 pairs of load uses – lw x6 to add x7, lw x8 to addi x9, and lw x11 to add x12. Four cycles are used to fill, hence 11+12+4=27 cycles.

Part D:

If we run the file b_hazard.s on RV32 5S architecture with forwarding and without hazard detection, the results will be accurate and there won't be any need for nops. The execution takes 12 cycles. All the dependencies in the code are between ALU operations and therefore can be forwarded on time.

In the case of every source operand, the forwarding logic selects the most recent data. In the case of add x7, x5, x6, rs1 (x5) is obtained from MEM/WB while rs2 (x6) is obtained from EX/MEM. In the case of sub x8, x7, x5, rs1 (x7) is obtained from EX/MEM while rs2 (x5) is obtained from register file since x5 has been written by that time. In the case of slli x9, x8, 2, rs1 (x8) is obtained from EX/MEM. In the case of or x11, x9, x10, rs1 (x9) is obtained from MEM/WB while rs2 (x10) is obtained from EX/MEM. In the case of add x12, x11, x7, rs1 (x11) is obtained from EX/MEM while rs2 (x7) is obtained from register file.

Using d_load.s with forwarding on RV32 5S architecture and without hazard detection leads to incorrect results. The result of a load will be ready in MEM stage; therefore, it won't be possible for forwarding to pass the result to the next instruction which needs it during its EX stage. Inserting one nop before each instruction that uses the result of the preceding load instruction solves this problem. Three nop instructions should be inserted after lw x6 and before add x7, after lw x8 and before addi x9, and after lw x11 and before add x12.

When running the original d_load.s code with the entire RV32 5S pipeline, it requires 18 cycles, including 3 stall cycles. Each load-use pair results in one stall cycle, which is half of the stall cycles required without forwarding in Part C, which was two stalls per load-use pair. The table below gives the cycles required per program per model, where "no" means it is incorrect.

Program	SS	No FW, no HZ (correct?)	No FW (correct?)	No HZ (correct?)	5S
b_hazard.s	8	12 (no)	22 (yes)	12 (yes)	12
d_load.s	11	15 (no)	27 (yes)	15 (no)	18

The forwarding unit also saves more cycles for d_load.s compared to the stall version. In moving from the former (with stalls but without forwarding) which has 27 cycles to the later one (pipeline), the savings is 9 cycles with most of the savings attributed to forwarding. On the other hand, the hazard unit maintains correctness of results where forwarding fails.

For the 5 stage clock of 270ps, the timing of control.s for RV32 5S is 26 cycles which is 26 * 270 = 7020 ps. Comparing with the single cycle timing of 12600 ps, the speedup factor is 12600 ÷ 7020 = 1.79. The 26 cycles above are hand-calculated hence verify using Ripes.

Part E:

These two instructions to be flushed when bne is taken in e_loop.s are “slli x13, x12, 1” and “addi x14, x13, 1”. They are the two instructions fetched by the pipeline after the bne instruction, and the outcome of the branch instruction is known in the EX stage.

There will be no penalty in case the branch instruction, bne, is not taken in the last iteration of the loop. The branch is predicted as not-taken by the pipeline, so it continues fetching PC+4. It takes zero cost to have a not-taken branch, while a taken branch incurs a cost of 2 flush cycles.

Cycles for n = 1, 2, 4, 8 are 18, 26, 42, and 74. Cycles per loop are 8, that is 5 instructions, stall cycle between lw x5 and add x12, and 2 flush cycles because of taken branch bne. The last loop requires 6 cycles, as this branch is not taken. Fixed part of the code is 12 cycles: 4 fill cycles, 6 prologue instructions, and 2 epilogue instructions. It means that cycles(n) = 8(n − 1) + 6 + 12 = 8n + 10. Thus, a = 8 and b = 10. For n = 16 cycles will be 8 × 16 + 10 = 138.

Since addi x20 and addi x21 in e sched.s are inserted after lw but before add, the add instruction no longer has to wait for the completion of the load operation. Thus, the iterations take 7 cycles each, the last one 5 cycles. The cycles(n) = 7n + 10; hence, a = 7 and b = 10, for n = 8 cycles are 66.

Reordering does not eliminate the cost of flush cycles because the bne operation is resolved in EX; therefore, two instructions after a taken branch will remain wrong-path instructions. All that can be done through reordering is changing the order of instructions inside the loop; hence, there will always be a branch misprediction with a cost of 2 cycles.

Part F:

The execution time for the original code f_kernel.s is 224 cycles. This code is executed in 128 instructions that includes 7 instructions before the loop. These cycles consist of 128 instructions, 16 stall loads, 76 flush cycles, and 4 fill cycles, which totals to 224.

The 16 stall cycles are generated due to each lw x5 instruction being immediately followed by a bge x5, x0, x0, which requires the loaded value. The 76 flush cycles are generated through three branches. The bge x5, x0 is taken in 10 cases of the positive values, requiring 20 cycles. The bge a1, x5 is taken in 13 cases of the value not being the maximum, requiring 26 cycles. The loop-back bne is taken 15 times, requiring 30 cycles. Bne is the most expensive branch because it works in every iteration.

The f opt.s optimization takes about 146 cycles, and thus satisfies the 150-cycle criteria for 9 points. The optimizations include two techniques. Firstly, all four values are loaded at the beginning of each iteration, and therefore, the branch instruction does not use a loaded value right after the load operation. Secondly, loop unrolling is done by a factor of four, and one pointer along with an end-pointer comparison is used. This results in the elimination of 38 instructions and 24 flush cycles, since the bne instruction is taken only three times instead of 15.

the branches were retained because of the absolute value and maximum functions. An absolute value without a branch would use 3 instructions per element, which would be expensive compared to the one with the branch. On the other hand, the maximum function without a branch would use roughly 5 instructions per element while the branch would be used only 13 times within 16 uses, therefore the branch would be more economical. The number 146 is my approximation, therefore, verify it on Ripes.